

DAA-WEEK 6

CH.JYOTHIRMAI

CH.SC.U4CSE24107

PRIMS ALGORITHM:

CODE:

```
#include <stdio.h>
#include <limits.h>
#define V 5
int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    return min_index;
}
void printMST(int parent[], int graph[V][V]) {
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++)
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
}
void primMST(int graph[V][V]) {
    int parent[V];
    int key[V];
    int mstSet[V];
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
    }
    printMST(parent, graph);
}
```

```

    printMST(parent, graph);
}
int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    primMST(graph);
    return 0;
}

```

OUTPUT:

```

Campus\SEM 4\DAA> ./pr
Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5

```

KRUSKALS ALGORITHM:

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#define V 4
#define E 5
struct Edge {
    int src, dest, weight;
};
struct Subset {
    int parent;
    int rank;
};
int find(struct Subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}
void Union(struct Subset subsets[], int x, int y) {
    int rootX = find(subsets, x);
    int rootY = find(subsets, y);

    if (subsets[rootX].rank < subsets[rootY].rank)
        subsets[rootX].parent = rootY;
    else if (subsets[rootX].rank > subsets[rootY].rank)
        subsets[rootY].parent = rootX;
    else {
        subsets[rootY].parent = rootX;
        subsets[rootX].rank++;
    }
}
int compare(const void* a, const void* b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

void KruskalMST(struct Edge edges[]) {
    struct Edge result[V];
    struct Subset subsets[V];
    qsort(edges, E, sizeof(edges[0]), compare);
    for (int v = 0; v < V; v++) {

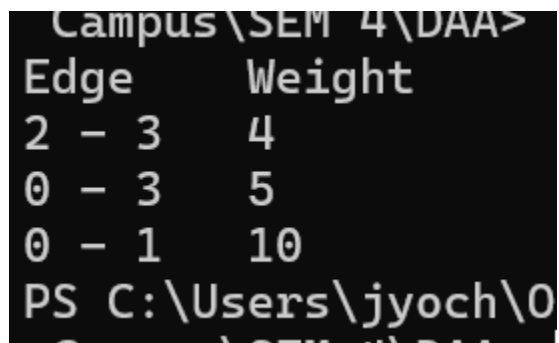
```

```

        subsets[v].parent = v;
        subsets[v].rank = 0;
    }
    int e = 0, i = 0;
    while (e < V - 1 && i < E) {
        struct Edge next = edges[i++];
        int x = find(subsets, next.src);
        int y = find(subsets, next.dest);
        if (x != y) {
            result[e++] = next;
            Union(subsets, x, y);
        }
    }
    printf("Edge \tWeight\n");
    for (i = 0; i < e; i++)
        printf("%d - %d \t%d\n", result[i].src, result[i].dest, result[i].weight);
}
int main() {
    struct Edge edges[E] = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    };
    KruskalMST(edges);
    return 0;
}

```

OUTPUT:



```

Campus\SEM 4\DAAS>
Edge      Weight
2 - 3      4
0 - 3      5
0 - 1     10
PS C:\Users\jyoch\0

```